

Hibernate

for configuration and setup refer to spring ORM (Hibernate) section in spring Boot Notebook.

@Entity - used to mark a java class as a persistent Entity its class whose object will be stored in database.
 - treat a class as database table.
 - Agar @Entity used kiya class Name ko upar to wo class table ki jaise used hoga class ka naam se table ka naam hoga.

- @Table(name = "employee") - custom table name rakhne ke liye.

ex. @Entity
 @Table(name = "user")
 public class User {
 // variables
 // getter, setter
 }

Note = class ka naam table ka naam hoga and variables ko name table me column hoga.

- @Id → to say it is primary key on column name.

- @GeneratedValue(strategy = GenerationType.IDENTITY)
 → used for auto incrementing column value.

- @Column(name = "student-name")
 - used on variable to say ye variable ka sath upar wala name mapped kardo for column name).
 - also used attribute length, unique) etc

- @@ one to one → one record in table "A" maps to one in table "B"

ex.

```

Table A
@Entity
public class Employee {
    id;
    @OneToOne
    private Address address;
    
```

```

Table B
@Entity
public class Address {
    addressid;
    private String city;
    
```

- @ one-to-many / manytoone
one student learn many subject

```

Table A
@Entity
public class Department {
    id;
    
```

```

Table B
@Entity
public class Employee {
    id;
    
```

```

@OneToMany(mappedBy = "depart")
private List<Employee> emp;
    
```

```

@ManyToOne
private Department dep;
    
```

Note → in this hibernate create relationship (Join) table if you not used mappedBy attribute

When we used mappedBy = "name" it not create any joint table.

many-to-many mapping

A student can take multiple courses and each course can have many students

Table A

@Entity

```
public class student {
    int id;
```

@manytomany

```
private List<course> courses;
```

Table B

@Entity

```
public class course {
    int id;
```

@manytomany(mappedBy = "courses")

```
private List<student> students;
```

mappedBy()

- used in bidirectional relationship.

- it indicates this side is not the owner of the relationship. The actual mapping is done on the other side.

@JoinColumn(name = "name")

- this will be foreign key in that table.

```
public class employee {
    int id;
```

```
public class Address {
    int id;
```

@onetoMany(mappedBy = "emp");

```
private Address add;
```

```
private Employee emp;
```

Note → above 'emp' says this employee side is not owning side and Address side is owning side which is 'emp' field in address class.

Example.

@ Entity

@ Table(name = "student")
public class student {

@ Id

@ GeneratedValue(strategy = GenerationType.IDENTITY)
private long id;

@ Column(name = "std_name", length = 100, unique = true)
private String name;

private String college;

@ Lob

private String about;

oneToOne Relationship

@Entity

```
public class Employee {
```

@Id

```
private int id;
```

@OneToOne

```
private Address add;
```

Explanation -

- ① hibernate will generate two tables - 1) Employee 1) Address
- ② in the employee table, hibernate will, Automatically add a foreign key column in Employee table
- ③ the side who own the relationship is Employee side table (jisme foreign key used hota hai)

imp - this by default, hibernate uses @JoinColumn

mappedby indicates this is not the owning side jo table
↓
diya hai wo field ownig hai
it does not managed foreign key

imp jo value mappedby = "employee" ne diya hai wo
non-owning side hai

For setting foreign key custom name we have to use
@JoinColumn(name = "address_id"). if not used then
mentioned for hibernate automatically add.

2 ways to configure hibernate

- ① xml based - refer ^{other} book (eclipse New method)
- ② java class based - refer eclipse

1) xml based

- ① Entity or pojo class
- ② hibernate.cfg.xml file.
- ③ App.java

- Step 1 → load xml file and build session factory
configuration
`cfg = new Configuration();`
`cfg.configure("xml file name");`
`SessionFactory sf = cfg.buildSessionFactory();`

- Step 2 → open session

`Session s = sf.openSession();`

- Step 3 → start transaction

`Transaction t = s.beginTransaction();`

- Step 4 → create and save entity.

`Entityclass obj = new Entityclass();`

set

Methods

`session.save(obj);`

- Step 5 → commit and close.

`try { session.save(obj);`

`t.commit();`

`} catch (Exception e) {`
`t.rollback();`

`- t`

Imp Session is always with used for CRUD operations methods-

⇒ insert operation

- save(), persist()

ex. obj.save(className);

⇒ select operation

→ get() → obj.get(Entity.class, id)

- load()

⇒ update operation - first student ki details save kar

- saveOrUpdate() - s.saveOrUpdate(Entity.class, id)

- update() - 72

⇒ Delete operation - pehle id batane ki konsi hai us

- delete() - 72

imp # Note.

Transaction method or object need when want to do only insert, delete and update operations.

#

Aspect-oriented programming (Aop)

- it is a programming paradigm used to separate cross-cutting concerns (like logging, security, transactions etc) from the main business logic.
- with Aop, you can add behaviour before, after or around method executions without changing the actual business code.

Term

- ① Aspect - A class that contains cross-cutting logic
- ② join point - A specific point in the execution of a program
- ③ Advice - the action taken at a particular join point
 - 1) before, after, around
- ④ pointcut - expression that matches join points.
ex - `* com.package.name.*.*(..)`
 *: any return type
 packageName : any class and method in the package
 (..): any parameters.
- ⑤ Weaving - Applying aspects to the target object

Advice.

- ① before - run before the method execution
- ② after - Run advice after the method execution
- ③ after-returning - Run advice after the method execution, only if the method completes successfully.
- ④ after-throwing - Run after method execution, only if the method exits by throwing an exception
- ⑤ around - Run advice before and after the advised method is invoked.

Spring Security

- it is a framework which provides various security features like authentication, authorization to create secure java enterprise applications.

① Authentication -

- verifying the identity of a user or system (eg. username and password, token-based (JWT)).
- confirms to who the user is.
- user ka username and password check karna. its called authentication.

② Authorization.

- Determining to whether the authenticated user has access to a ~~ser~~ resource.
- Determines what the user is allowed to do.
- ek baar user authenticate ho gaya to wo konsi method or section ko used karega is called authorization.
- ex. user authenticate hone ke baad wo user ya admin section ko access kar sakta hai ye batana means authorization.

How to create project and configure

- 1) create project
- 2) add security dependency in pom.xml.
- 3) create config package and securityConfig.java class

@Configuration - to tell this is config class

@EnableWebSecurity - Enable spring web security support

Verbal Ability

Authenticate and authorize inMemory.

① create bean in config file of a user details.
means create users (different users) using userDetails
service. (user details here)

For full code refer STS

② create bean of PasswordEncoder.

③ create bean of SecurityFilterChain.

- @PreAuthorize

- Expression-based method security

- used jab koi method ye Role wala hi access
kar sakta hai, agar (admin) likha to sirf
wo method admin hi use kr ya authorized kr payega

- @EnableMethodSecurity(prePostEnabled = true)

- used to with only preAuthorize() method

- to preauthorize ka method use krne ke liye isko
enable krna hoga.

Authentication and Authorization using database

steps only for security code: (SecurityConfig package)

- ① create CustomUser class → implements UserDetails
- ② CustomUserDetailsService class → implements UserDetailsService
- ③ create SecurityConfig class →

Now creates beans

- SecurityFilterChain bean

- PasswordEncoder bean

- create DaoProvider bean → UserDetails obj, Password encoder

requestMatchers("/").permitAll()

this means Anyone (even not logged in) can access to '/' URI - usually your index page that you mapped in Controller

* .anyRequest().authenticated()

this means all other URI (lib / dashboard, / profile, etc) request to user to be logged in.